



MASTERCLASS · TALLER

Spec-Driven Development

Dirigir a un agente de IA con una especificación, no con prompts sueltos

Hugo Menéndez Montes · AI Engineering Program Lead

¿QUÉ VAMOS A HACER HOY?

Salir sabiendo escribir una **spec** y usarla para **dirigir a un agente de IA** en un proyecto real.

20'

Teoría

Del vibe coding a SDD, los niveles, las piezas del flujo y EARS.

35'

Demo en vivo

Una funcionalidad nueva en el RAG de la agenda cultural, fase a fase.

45'

Práctica

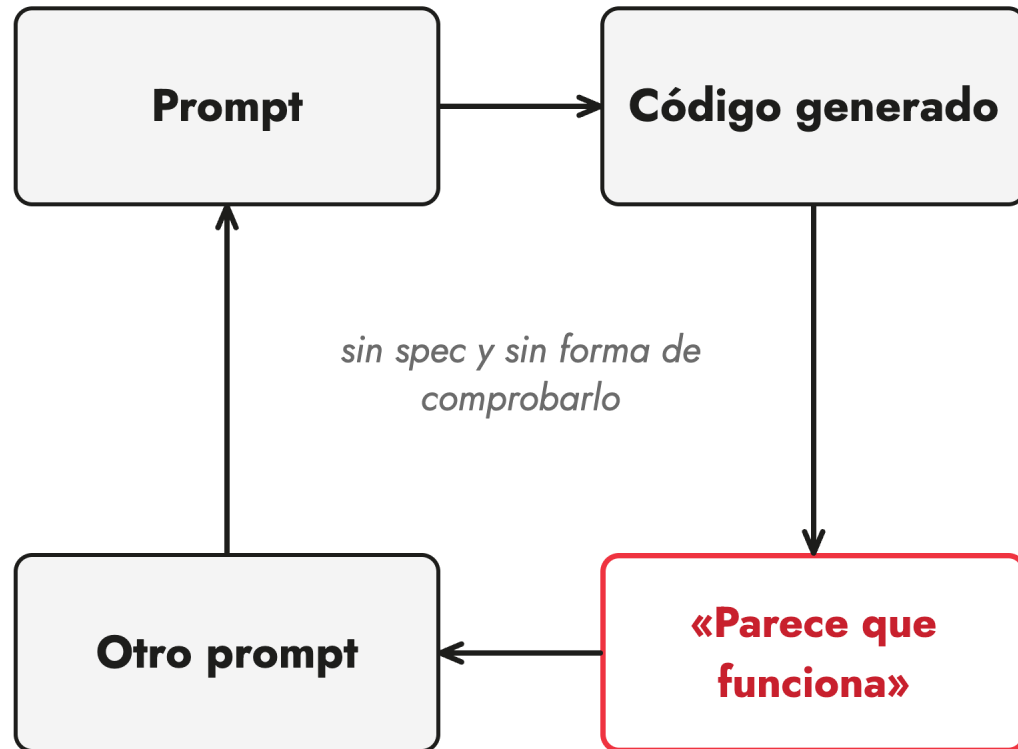
El mismo flujo sobre tu propio proyecto RAG.

10'

Puesta en común

Qué encontré tu clarificación y qué te llevas.

DEL VIBE CODING...



Para un prototipo va de maravilla. Se rompe cuando el proyecto crece:

- ⚠ El agente añade cosas que nadie pidió.
- ⚠ Arregla un bug y rompe otra cosa.
- ⚠ Nadie sabe por qué el código es así.
- ⚠ Cada sesión con el agente empieza de cero.

...A SPEC-DRIVEN DEVELOPMENT

La especificación es la **fuentes de verdad**.
El código es un subproducto.

Vive en el repo

Legible por personas y por el agente. No es un prompt que se pierde en el chat.

Primero la spec

Si cambia el comportamiento, se edita la spec y después el código.

Se comprueba

Cada requisito tiene su test o su comprobación manual.

No sustituye a TDD ni a BDD: los engloba. La spec genera los criterios que luego se convierten en tests.

El problema no es el agente.

Es darle instrucciones ambiguas y no tener forma de comprobar lo que hace.

NIVELES DE MADUREZ

1

Spec-First

La spec se escribe antes de pedir código, pero vive en el chat.

Límite: se desconecta del código al terminar.

2

Spec-Anchored

La spec vive en el repo y el agente la implementa por fases.

Límite: la misma spec puede dar implementaciones distintas.

Hoy trabajamos aquí

3

Spec-as-Source

La spec es la fuente canónica; el código se regenera cuando cambia.

Límite: regenerar puede sorprender.

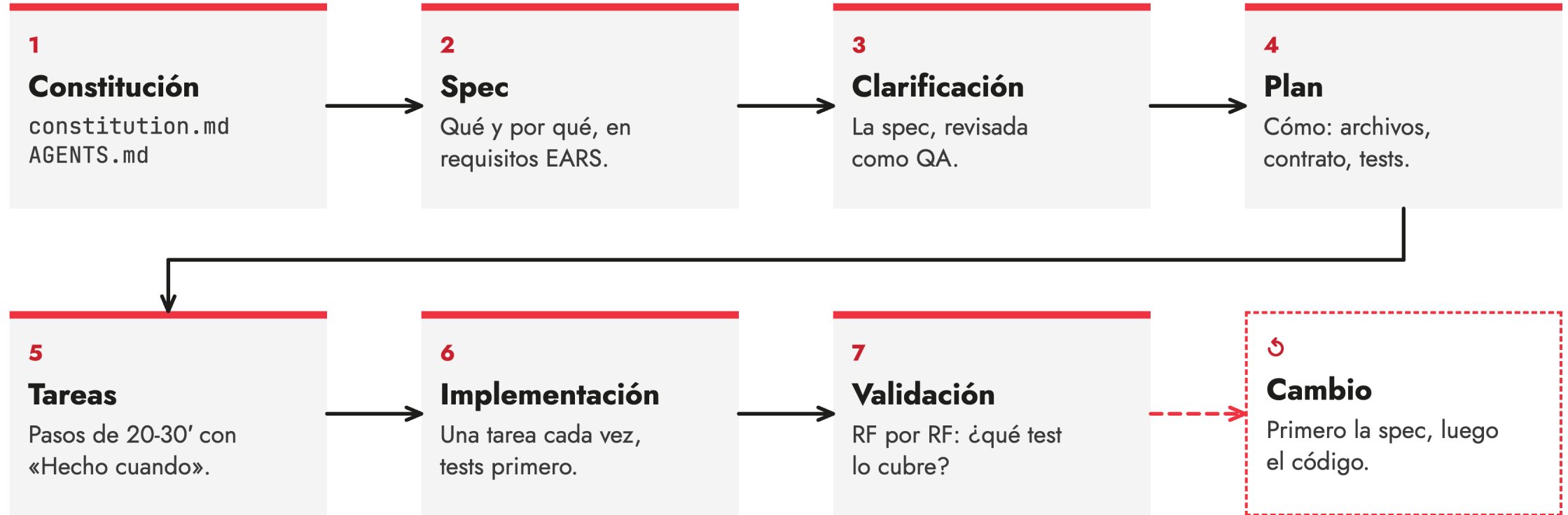
4

Spec-to-Application

La spec es un modelo formal y se compila, sin LLM.

Límite: determinista, pero rígido de montar.

EL FLUJO SDD



Entre fase y fase, una persona revisa y aprueba. **Tú decides, el agente ejecuta.**

EL CONTEXTO FIJO DEL AGENTE

docs/constitution.md

Principios innegociables del proyecto

La spec manda. Si falta una decisión, se para y se pregunta.

Anclado al corpus. Si no hay evidencia, se abstiene.

Tests como puerta, sin API. Sin pytest en verde no se avanza.

AGENTS.md

El system prompt del proyecto

Mapa del código y comandos, marcando los que gastan API.

Fuera de la implementación no se escribe código.

Una tarea cada vez, tests primero, y te paras.

El agente los lee antes de cada fase: se escriben una vez y se aplican siempre.

EARS: REQUISITOS COMPROBABLES

Easy Approach to Requirements Syntax · Alistair Mavin (Rolls-Royce), 2009 · palabras clave en orden fijo · numerados RF-1, RF-2...

Patrón	Plantilla	Ejemplo en el RAG de la agenda cultural
Ubicuo	EL SISTEMA <hará>	EL SISTEMA mostrará las fuentes usadas debajo de cada respuesta.
Evento	CUANDO <disparador>, EL SISTEMA <hará>	CUANDO se pregunte con -ask, EL SISTEMA mostrará la respuesta y sus fuentes.
No deseado	SI <condición>, ENTONCES EL SISTEMA <hará>	SI la pregunta está vacía, ENTONCES EL SISTEMA mostrará un error sin llamar al LLM.
Estado	MIENTRAS <estado>, EL SISTEMA <hará>	MIENTRAS el índice esté vacío, EL SISTEMA indicará que hay que ejecutar -index.
Opcional	DONDE <característica>, EL SISTEMA <hará>	DONDE haya un umbral de relevancia, EL SISTEMA descartará los fragmentos que lo superen.

EJERCICIO: REESCRÍBELO EN EARS



«El asistente tiene que responder bien a las preguntas raras.»

¿Qué es «bien»? ¿Qué es «rara»? ¿Cómo lo comprobarías?

RF-a: SI la pregunta está vacía, **ENTONCES EL SISTEMA** mostrará un error sin llamar al LLM.

RF-b: SI la pregunta supera 2000 caracteres, **ENTONCES EL SISTEMA** la rechazará con un mensaje de error.

RF-c: SI ningún fragmento recuperado es relevante, **ENTONCES EL SISTEMA** responderá que no tiene información, sin llamar al LLM.

← la de la demo

CÓMO SE ESCRIBE UN BUEN REQUISITO

1**Una frase, un comportamiento**

Si aparece un «y» que une dos cosas, son dos RF.

2**Verificable**

Si no se te ocurre el test o el comando que lo comprueba, está mal escrito.

3**Nada de adjetivos sin número**

«Rápido» → «en menos de 2 s». «Corto» → «como mucho 3 frases».

4**Solo el qué y el porqué**

Ni archivos, ni funciones, ni librerías: eso va en el plan.

5**Lo que no sabes, se marca**

[NECESITA ACLARACIÓN]: un hueco visible es información; una suposición silenciosa es deuda.

CONSTRUIR AGENTES VS. DIRIGIRLOS

	Construir un agente	Dirigir un agente con SDD
Qué se construye	El propio agente: tools, memoria, bucle	Software, con un agente ya hecho como motor
Dónde se quita la ambigüedad	En el schema de cada tool y en el system prompt	En la spec, antes de escribir código
Cómo se verifica	Evals del agente, tests de tool calling	Requisito por requisito contra la spec
Papel del agente	Autónomo: decide qué tool usar y cuándo	Semidirigido: sigue una spec y un plan acordados



AGENTS.md + la spec son el system prompt del proyecto. **Los RF en EARS** son el schema estricto, pero de los requisitos.

35 MINUTOS

DEMO EN VIVO

Abstención por baja relevancia en el RAG de la agenda cultural de Madrid

```
python main.py --ask "¿Cuál es la capital de Francia?"
```

LOS COMANDOS /SDD-*

Fase	Comando en Copilot Chat	Qué deja
Constitución y contexto	<code>/sdd-constitucion · /sdd-contexto</code>	constitution.md · AGENTS.md
Línea base (proyecto existente)	<code>/sdd-spec-inversa</code>	specs/000-.../spec.md
Especificación	<code>/sdd-spec <idea></code>	specs/NNN-.../spec.md
Clarificación	<code>/sdd-clarificar</code>	la misma spec, sin dudas abiertas
Plan	<code>/sdd-plan</code>	plan.md
Tareas	<code>/sdd-tareas</code>	tasks.md
Implementación	<code>/sdd-implementar T1</code>	código + tests, tarea marcada [x]
Validación	<code>/sdd-validar</code>	validacion.md
Cambio	<code>/sdd-cambio <requisito></code>	diff de la spec, sin tocar código

¿No aparecen al escribir «/»? Abre el `.prompt.md` de la fase, copia el texto y pégalo en el chat en modo Agent.

45 MINUTOS

PRÁCTICA

SDD sobre tu propio proyecto RAG: una funcionalidad pequeña, de la idea a la primera tarea con tests en verde.

```
Masterclasses/MC_05_SDD/Plantilla/README.md
```

TUS 45 MINUTOS



Spec aclarada + una tarea en verde = taller hecho.

PARA SEGUIR APRENDIENDO

mouredev/hello-sdd

Curso de SDD desde cero: plantillas y un proyecto completo, paso a paso.

GitHub Spec Kit

El toolkit de referencia de SDD, con comandos `/speckit.*` para muchos agentes.

EARS, de Alistair Mayin

La notación de requisitos contada por su creador.

Prompt files en VS Code

Cómo crear tus propios comandos para Copilot.



En el repo: **SDD_workshop**, la demo completa con cada fase resuelta, y **Plantilla**, para aplicarlo a tu proyecto.

